SECURITY INVESTIGATION REPORT

Student ID: B00173541,Sofiia Prymak

GitHub: https://github.com/Advanced-Scripting-2026/asc2026-project-assessment-SofiiaPrymak

Оглавление

Introduction *3*

How the system works *3*

Brute-force detection *4*

PowerShell analysis *5*

IOC matching *5*

Results *5*

Conclusion *7*

Introduction

In modern cybersecurity systems, analysis is the most important thing. Log analysis is one of the important tasks. Security analysts often process a huge amount of data that can come directly from device servers or from other offers. Performing it manually will take a very long time and will also increase the risk of missing a critical threat. This is why automated tools similar to those in my project are very often used in real SOC systems. My project also helped me understand how attackers behave in real life. For example, brute force attacks often come from very different IP addresses and literally scattered across different places and the world, which makes it more difficult to locate them

In this project, I developed a security log analysis tool that helps detect suspicious activity in a system. Essentially, it's a small SOC analysis tool that automatically checks logs and identifies signs of attacks.I worked on and developed a tool for analyzing security logs that helps detect suspicious activity in the system. The goal of my project and tool is to simulate the basic process of security operations center (SOC), in short, where system logs are analyzed to identify potential attacks and threats, or whether it could be abnormal behavior.

I worked with several types of logs:

• authentication log (auth.log) Contains login attempts, both successful and unsuccessful.

• access log (access.log), Records general system or web service access actions.

• PowerShell log, Contains executed PowerShell commands.

• indicator of compromise (IOC) files. Contains known malicious indicators (IP addresses, domains, signatures).

The main idea of the project is to automate analysis to avoid manually sifting through thousands of lines.

How the system works

The system is written in Python and operates on a simple principle: it reads log files and looks for specific signs of attacks.

The tool works according to my pipeline stage. First, it reads authentication logs and extracts unnecessary and failed login attempts, then analyzes PowerShell logs to identify malicious commands. After that, it compares all the collected data with a list of IOC indicators. The results are copied and saved in a new file, which can then be analyzed if necessary. This approach is important because it reflects the actual process

of SOC operation in a professional environment. The data is processed in stages, which includes collection, analysis, and finally reporting

I used three main analysis methods:

1. Searching for brute-force attacks (many unsuccessful logins)

2. Searching for suspicious PowerShell commands

3. Comparing logs with IOC (known threats)

If the system finds a match, it records it in a report.

Brute-force detection

Brute force is all I can say; this type of attack is the most common compared to other types of cyber attacks. The attacker uses automated scripts or there may also be bots that can try thousands of passwords until they find the right one. Such attacks usually create a large number of login errors in a very short period of time.

Speaking about my project, in my implementation I used a simple but very effective rule: any IP address with more than 5 incorrect login attempts is considered strange and suspicious, although this threshold is simple, it will work well for executing obvious attack patterns.

The most dangerous IP address in the data was 24.151.103.17 with 157 login attempts. I can say this is almost certainly an automated attack because a normal user cannot physically make so many errors in a row in real SOC systems. This can be improved by adding time analysis. I'll give an example of the number of attempts per minute, geolocation of IP addresses, and correlation with successful logins.

One of the main features is brute-force attack detection

I scan the auth.log file for “Failed password” entries and count how many times each IP tries to log in.

If an IP has more than 5 attempts, it is marked as suspicious.

Results:

- 24.151.103.17 → 157 attempts (most dangerous)
- 181.25.206.27 → 12 attempts
- 85.245.107.41 → 11 attempts

This shows that someone was likely trying to guess passwords automatically.

PowerShell analysis

PowerShell is a powerful tool that is often used by attackers; malicious scripts can use encoded commands to hide their real activity and behavior from detection systems.

For example, (encodedcommand) is often used to obfuscate malicious scripts, and (invoke webrequest) can be used to download malware from external servers. My method in the detection project is based on searching for keywords. Despite my simplicity, it is very effective in identifying known malicious patterns. However, it can also give false positives if administrative scripts use something similar or the same commands.

The second part of the analysis is checking PowerShell logs.

PowerShell is often used in attacks because it can be used to run hidden commands.

I searched for the following keywords:

• EncodedCommand

• DownloadString

• Invoke-WebRequest

• ExecutionPolicy Bypass

If such commands appear, it could be a sign of an attack.

IOC matching

IOC is a list of known malicious indicators (e.g., IP addresses or domains).

My system compares this data with logs.

In my system, IOCs are loaded from a file and compared against all log lines from the authentication, access, and PowerShell logs. If a match is found, it is written to the final report.

This method is useful for detecting known attacks, but it cannot detect new or unknown threats (zero day attacks). Therefore, in real-world systems, IOC is typically used in conjunction with behavioral analysis.

If there's a match, it means the system may have interacted with the threat.

Results

The tool is executed using the following command:

1.Tool execution in terminal

```
Report generated!
sofiiaprymak@Mac Project_folder % python3 run.py
Report generated!
sofiiaprymak@Mac Project_folder %
```

2.Detected results showing brute force attempts

After running the program, the following issues were found:

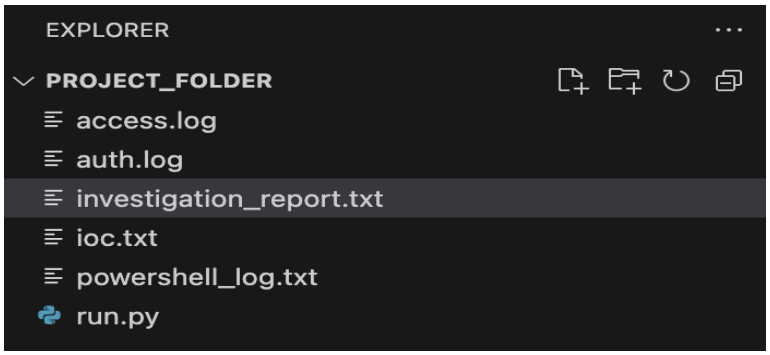
investigation_report.txt

```
SECURITY REPORT

Failed Login Attempts:
85.245.107.41 -> 11 attempts
181.25.206.27 -> 12 attempts
24.151.103.17 -> 157 attempts
181.25.201.155 -> 8 attempts
122.189.198.238 -> 6 attempts
60.187.118.40 -> 6 attempts
42.184.142.151 -> 6 attempts
82.64.2.59 -> 6 attempts
```

Detected brute force login attempts from multiple ip addresses exceeding the threshold the ip 24.151.103.17 is identified as the most critical threat due to the extremely high number of failed login attempts

3. generated report files




```
10.223.157.186 - - [15/Jul/2009:14:58:59 -0700] "GET / HTTP/1.1" 403 202
10.223.157.186 - - [15/Jul/2009:14:58:59 -0700] "GET /favicon.ico HTTP/1.1" 404 209
10.223.157.186 - - [15/Jul/2009:15:50:35 -0700] "GET / HTTP/1.1" 200 9157
10.223.157.186 - - [15/Jul/2009:15:50:35 -0700] "GET /assets/js/lowpro.js HTTP/1.1" 200 10469
10.223.157.186 - - [15/Jul/2009:15:50:35 -0700] "GET /assets/css/reset.css HTTP/1.1" 200 1014
10.223.157.186 - - [15/Jul/2009:15:50:35 -0700] "GET /assets/css/960.css HTTP/1.1" 200 6206
10.223.157.186 - - [15/Jul/2009:15:50:35 -0700] "GET /assets/css/the-associates.css HTTP/1.1" 200 15779
10.223.157.186 - - [15/Jul/2009:15:50:35 -0700] "GET /assets/js/the-associates.js HTTP/1.1" 200 4492
10.223.157.186 - - [15/Jul/2009:15:50:35 -0700] "GET /assets/js/lightbox.js HTTP/1.1" 200 25960
10.223.157.186 - - [15/Jul/2009:15:50:36 -0700] "GET /assets/img/search-button.gif HTTP/1.1" 200 168
10.223.157.186 - - [15/Jul/2009:15:50:36 -0700] "GET /assets/img/dummy/secondary-news-3.jpg HTTP/1.1" 200 5604
10.223.157.186 - - [15/Jul/2009:15:50:36 -0700] "GET /assets/img/dummy/primary-news-1.jpg HTTP/1.1" 200 10556
10.223.157.186 - - [15/Jul/2009:15:50:36 -0700] "GET /assets/img/dummy/primary-news-2.jpg HTTP/1.1" 200 9925
10.223.157.186 - - [15/Jul/2009:15:50:36 -0700] "GET /assets/img/closelabel.gif HTTP/1.1" 200 979
10.223.157.186 - - [15/Jul/2009:15:50:36 -0700] "GET /assets/img/home-logo.png HTTP/1.1" 200 3892
10.223.157.186 - - [15/Jul/2009:15:50:36 -0700] "GET /assets/img/dummy/secondary-news-2.jpg HTTP/1.1" 200 5397
10.223.157.186 - - [15/Jul/2009:15:50:36 -0700] "GET /assets/img/loading.gif HTTP/1.1" 200 2767
10.223.157.186 - - [15/Jul/2009:15:50:36 -0700] "GET /assets/img/dummy/secondary-news-4.jpg HTTP/1.1" 200 5766
10.223.157.186 - - [15/Jul/2009:15:50:36 -0700] "GET /assets/img/home-media-block-placeholder.jpg HTTP/1.1" 20
10.223.157.186 - - [15/Jul/2009:15:50:37 -0700] "GET /assets/img/dummy/secondary-news-1.jpg HTTP/1.1" 200 5766
10.223.157.186 - - [15/Jul/2009:15:50:37 -0700] "GET /assets/swf/home-media-block.swf HTTP/1.1" 200 123884
```

• a large number of brute-force attacks

• suspicious PowerShell commands

• matches with IOCs

The most serious finding was IP 24.151.103.17 with 157 login attempts.

This is almost certainly an automated attack.

Conclusion

This project helped me understand how security analysis works in real SOC systems.

Overall, this project demonstrates the importance of automation in cyber security, as manual log analysis is ineffective and prone to human error, while automated tools can quickly identify suspicious patterns. Future developments may also include the implementation of machine learning, which will study normal system behavior and can also identify deviations. A useful improvement would be the creation of visual dashboards that will help analysts better understand the structure of all attacks.

I learned:

• log analysis

• attack detection

• process automation

• working with Python for cybersecurity

The tool can be improved by adding:

• machine learning

• graphs

• real-time monitoring